

QUESTION ONE

a) Define the term "Common Language Runtime (CLR)" and explain FOUR services it provides to a VB.NET application. (3 marks)

Answer:

The **Common Language Runtime (CLR)** is the execution environment in the .NET Framework that manages and runs VB.NET applications.

Four services provided by the CLR are:

1. **Memory management** – Allocates and manages memory used by the application.
 2. **Garbage collection** – Automatically removes objects that are no longer being used.
 3. **Exception handling** – Detects and handles runtime errors.
 4. **Security** – Provides security checks when running applications.
-

b) Explain the difference between a structure and a class in VB.NET. Demonstrate each with a code example showing when you would use each. (3 marks)

Answer:

A **Structure** is a **value type**, while a **Class** is a **reference type**.

- A **Structure** is suitable for small and simple data, such as coordinates.
- A **Class** is suitable for complex objects that contain data and methods.

Structure example

Structure Point

Public X As Integer

Public Y As Integer

End Structure

Module Program

Sub Main()

Dim p As New Point

p.X = 10

p.Y = 20

Console.WriteLine("X = " & p.X)

Console.WriteLine("Y = " & p.Y)

End Sub

End Module

Use: A structure is useful for simple related values such as a point (X, Y).

Class example

Class Student

Public Name As String

Public Sub Display()

Console.WriteLine("Student: " & Name)

End Sub

End Class

Module Program

Sub Main()

Dim student1 As New Student()

student1.Name = "John"

student1.Display()

End Sub

End Module

Use: A class is useful for objects such as students that have data and behavior.

c) Explain the purpose of the Using statement in VB.NET. Demonstrate its usage with a code example that reads data from a text file. (3 marks)

Answer:

The **Using statement** is used to automatically release or dispose of resources such as files, streams, and database connections after they have been used.

Example

Imports System.IO

Module Program

Sub Main()

Using reader As New StreamReader("data.txt")

While Not reader.EndOfStream

Console.WriteLine(reader.ReadLine())

End While

End Using

End Sub

End Module

Working:

- StreamReader opens the file.
- ReadLine() reads each line.
- Using automatically closes and releases the file resource when the block ends.

d) Define the term "assembly manifest" and list THREE types of information stored in it. (3 marks)

Answer:

An **assembly manifest** is information stored inside a .NET assembly that describes the assembly and its contents.

Three types of information stored in an assembly manifest are:

1. **Assembly name and version**
2. **Referenced assemblies**
3. **Security and identity information**

e) Explain the difference between an abstract class and an interface in VB.NET. Demonstrate a scenario where using an interface would be better than using an abstract class. (3 marks)

Answer:

An **abstract class** is a base class that can contain both implemented methods and abstract methods.

An **interface** defines a contract that a class must follow.

An interface is better when **different and unrelated classes need to perform the same function**.

Example

```
Interface IPrintable  
    Sub Print()  
End Interface
```

```
Class Student  
    Implements IPrintable
```

```
Public Sub Print() Implements IPrintable.Print
    Console.WriteLine("Printing student details")
End Sub
End Class
```

```
Class Invoice
    Implements IPrintable
```

```
Public Sub Print() Implements IPrintable.Print
    Console.WriteLine("Printing invoice")
End Sub
End Class
```

Why interface is better here:

Both Student and Invoice need a Print() function, but they are different types of objects.

f) Explain the purpose of the WithEvents keyword in VB.NET. Define the relationship between an event source, an event handler, and a delegate. (3 marks)

Answer:

The **WithEvents** keyword allows an object to receive and respond to events raised by that object.

Relationship:

- **Event source:** The object that raises an event, for example a Button.
- **Event handler:** The method that runs when the event occurs.
- **Delegate:** Defines the method signature used to connect the event to its handler.

Example

```
Private WithEvents btnSave As Button
```

```
Private Sub btnSave_Click(sender As Object, e As EventArgs) _
    Handles btnSave.Click
```

```
    MessageBox.Show("Button clicked")
```

```
End Sub
```

Here:

- btnSave = event source
- btnSave_Click = event handler
- The event delegate connects the Click event to the handler.

g) State TWO benefits of using namespaces in a VB.NET project and give TWO examples of commonly used namespaces in Windows Forms programming. (3 marks)

Answer:

Two benefits of namespaces:

1. They **organize related classes and code**.
2. They **prevent naming conflicts** between classes with the same name.

Two commonly used Windows Forms namespaces:

1. System.Windows.Forms
2. System.Drawing

h) Build a complete VB.NET code example showing: (5 marks)

- A class called BankAccount with a private field `_balance`
- A public property `Balance` with Get and Set accessors that validates the amount cannot be negative
- A method called `Deposit` that adds to the balance and uses the property
- A method called `Withdraw` that subtracts from the balance if sufficient funds exist

Answer:

```
Public Class BankAccount
```

```
    Private _balance As Decimal
```

```
    Public Property Balance As Decimal
```

```
        Get
```

```
            Return _balance
```

```
        End Get
```

```
        Set(value As Decimal)
```

```
            If value >= 0 Then
```

```
                _balance = value
```

```
            Else
```

```
                Throw New ArgumentException("Balance cannot be negative.")
```

```
            End If
```

```
        End Set
```

```
    End Property
```

```
    Public Sub Deposit(amount As Decimal)
```

```
        If amount > 0 Then
```

```
            Balance = Balance + amount
```

```

Else
    Console.WriteLine("Invalid deposit amount.")
End If

End Sub

Public Sub Withdraw(amount As Decimal)

    If amount > 0 AndAlso amount <= Balance Then
        Balance = Balance - amount
    Else
        Console.WriteLine("Insufficient funds or invalid amount.")
    End If

End Sub

End Class

```

Working:

Suppose:

```

Dim account As New BankAccount()

account.Balance = 1000
account.Deposit(500)
account.Withdraw(200)

Console.WriteLine(account.Balance)

```

Calculation:

```

Initial Balance = 1000
Deposit = 500
Balance = 1000 + 500 = 1500

Withdraw = 200
Balance = 1500 - 200 = 1300

```

Output:

1300

i) Demonstrate a VB.NET code snippet that shows: (4 marks)

- A Button control declared with WithEvents
- An event handler using the Handles keyword
- The event handler updating a Label control with the current date and time when clicked
- Explain what would happen if the Handles keyword was omitted

Answer:

Public Class Form1

```
Private WithEvents btnDateTime As New Button()  
Private lblDateTime As New Label()
```

```
Private Sub btnDateTime_Click(sender As Object, e As EventArgs) _  
    Handles btnDateTime.Click
```

```
    lblDateTime.Text = DateTime.Now.ToString()
```

```
End Sub
```

End Class

Working:

When the button is clicked:

Button Click

↓

btnDateTime_Click()

↓

DateTime.Now

↓

Label displays current date and time

If Handles is omitted:

The method will **not automatically be connected** to the button's Click event. Therefore, clicking the button will not run the method unless the event is connected manually using AddHandler.

j) Build a VB.NET code example showing: (4 marks)

- An abstract class called Shape with abstract methods CalculateArea() and CalculatePerimeter()
- A derived class called Rectangle that implements both methods
- A derived class called Circle that implements both methods
- Explain TWO differences between the abstract class you created and an interface

Answer:

Public MustInherit Class Shape

```
Public MustOverride Function CalculateArea() As Double
```

```
Public MustOverride Function CalculatePerimeter() As Double
```

End Class

Public Class Rectangle

```
Inherits Shape
```

```

Private Width As Double
Private Height As Double

Public Sub New(width As Double, height As Double)
    Me.Width = width
    Me.Height = height
End Sub

Public Overrides Function CalculateArea() As Double
    Return Width * Height
End Function

Public Overrides Function CalculatePerimeter() As Double
    Return 2 * (Width + Height)
End Function

End Class

```

```

Public Class Circle
    Inherits Shape

    Private Radius As Double

    Public Sub New(radius As Double)
        Me.Radius = radius
    End Sub

    Public Overrides Function CalculateArea() As Double
        Return Math.PI * Radius * Radius
    End Function

    Public Overrides Function CalculatePerimeter() As Double
        Return 2 * Math.PI * Radius
    End Function

End Class

```

Working:

For a rectangle with width 10 and height 5:

$$\begin{aligned}
 \text{Area} &= \text{Width} \times \text{Height} \\
 &= 10 \times 5 \\
 &= 50
 \end{aligned}$$

$$\begin{aligned}
 \text{Perimeter} &= 2 \times (\text{Width} + \text{Height}) \\
 &= 2 \times (10 + 5) \\
 &= 30
 \end{aligned}$$

For a circle with radius 7:

$$\begin{aligned}
 \text{Area} &= \pi r^2 \\
 &= \pi \times 7^2 \\
 &= 153.94 \text{ approximately}
 \end{aligned}$$

Perimeter = $2\pi r$
= $2 \times \pi \times 7$
= 43.98 approximately

Two differences:

1. An **abstract class** can contain both implemented and abstract methods, while an **interface** mainly defines members that implementing classes must provide.
2. A class can inherit from only **one base/abstract class**, but it can implement **multiple interfaces**.

k) Build a complete VB.NET code example that demonstrates the use of a Try...Catch...Finally block to handle file operations. Include: (5 marks)

- A Try block that attempts to read a file
- A Catch block that handles FileNotFoundException
- A Catch block that handles IOException
- A Finally block that displays a cleanup message
- Explain what happens if an exception is not handled in a VB.NET application

Answer:

```
Imports System.IO
```

```
Module FileExample
```

```
Sub Main()
```

```
Try
```

```
Dim content As String = File.ReadAllText("data.txt")  
Console.WriteLine(content)
```

```
Catch ex As FileNotFoundException
```

```
Console.WriteLine("Error: File was not found.")
```

```
Catch ex As IOException
```

```
Console.WriteLine("Error reading the file.")
```

```
Finally
```

```
Console.WriteLine("Cleanup completed.")
```

```
End Try
```

```
End Sub
```

```
End Module
```

Working:

If the file does not exist:

```
Try
↓
File.ReadAllText()
↓
FileNotFoundException
↓
First Catch executes
↓
Finally executes
```

If another file-related error occurs:

```
Try
↓
IOException
↓
Second Catch executes
↓
Finally executes
```

The Finally block executes whether an exception occurs or not.

If an exception is not handled:

The application may **terminate unexpectedly**, and the .NET runtime reports the unhandled exception.

I) Build a namespace structure for a library management system with the following: (5 marks)

- Library.Books namespace containing a Book class with properties: BookID, Title, Author, ISBN, IsAvailable
- Library.Members namespace containing a Member class with properties: MemberID, FullName, Email, Phone
- Library.Loans namespace containing a Loan class with properties: LoanID, BookID, MemberID, LoanDate, ReturnDate
- In a Windows Form, import all namespaces, create instances of each class, and display sample data in a ListBox

Answer:

Book.vb

Namespace Library.Books

Public Class Book

```
Public Property BookID As Integer
Public Property Title As String
Public Property Author As String
Public Property ISBN As String
Public Property IsAvailable As Boolean
```

```
End Class
```

```
End Namespace
```

```
Member.vb
```

```
Namespace Library.Members
```

```
Public Class Member
```

```
Public Property MemberID As Integer
Public Property FullName As String
Public Property Email As String
Public Property Phone As String
```

```
End Class
```

```
End Namespace
```

```
Loan.vb
```

```
Namespace Library.Loans
```

```
Public Class Loan
```

```
Public Property LoanID As Integer
Public Property BookID As Integer
Public Property MemberID As Integer
Public Property LoanDate As DateTime
Public Property ReturnDate As DateTime?
```

```
End Class
```

```
End Namespace
```

```
Windows Form
```

```
Imports Library.Books
```

```
Imports Library.Members
```

```
Imports Library.Loans
```

```
Public Class Form1
```

```
Private Sub Form1_Load(sender As Object, e As EventArgs) _
    Handles MyBase.Load
```

```
Dim book As New Book With {
    .BookID = 1,
    .Title = "VB.NET Programming",
    .Author = "John Smith",
    .ISBN = "123456789",
    .IsAvailable = True
}
```

```
Dim member As New Member With {
    .MemberID = 101,
```

```

        .FullName = "Mary Wambui",
        .Email = "mary@gmail.com",
        .Phone = "0712345678"
    }

    Dim loan As New Loan With {
        .LoanID = 500,
        .BookID = 1,
        .MemberID = 101,
        .LoanDate = DateTime.Today,
        .ReturnDate = Nothing
    }

    ListBox1.Items.Add("BOOK: " & book.Title)
    ListBox1.Items.Add("Author: " & book.Author)
    ListBox1.Items.Add("Available: " & book.IsAvailable)

    ListBox1.Items.Add("MEMBER: " & member.FullName)
    ListBox1.Items.Add("Email: " & member.Email)
    ListBox1.Items.Add("Phone: " & member.Phone)

    ListBox1.Items.Add("LOAN ID: " & loan.LoanID)
    ListBox1.Items.Add("Loan Date: " &
        loan.LoanDate.ToShortDateString())

End Sub

```

End Class

Working:

The program:

```

Import namespaces
↓
Create Book object
↓
Create Member object
↓
Create Loan object
↓
Add their details to ListBox

```

m) Build a VB.NET code example demonstrating delegates: (5 marks)

- Declare a delegate called Calculation that takes two integers and returns an integer
- Create methods Add, Subtract, and Multiply that match the delegate signature
- Demonstrate how to assign each method to the delegate and invoke it
- Demonstrate a multicast delegate that calls multiple methods

Answer:

Module DelegateExample

```
'Declare delegate
Delegate Function Calculation(a As Integer, b As Integer) As Integer
```

```
'Methods
Function Add(a As Integer, b As Integer) As Integer
    Return a + b
End Function
```

```
Function Subtract(a As Integer, b As Integer) As Integer
    Return a - b
End Function
```

```
Function Multiply(a As Integer, b As Integer) As Integer
    Return a * b
End Function
```

```
Sub Main()
```

```
    Dim calc As Calculation
```

```
    'Assign Add method
    calc = AddressOf Add
    Console.WriteLine("Addition = " & calc(10, 5))
```

```
    'Assign Subtract method
    calc = AddressOf Subtract
    Console.WriteLine("Subtraction = " & calc(10, 5))
```

```
    'Assign Multiply method
    calc = AddressOf Multiply
    Console.WriteLine("Multiplication = " & calc(10, 5))
```

```
    'Multicast delegate
    Dim multiCalc As Calculation
```

```
    multiCalc = AddressOf Add
    multiCalc = [Delegate].Combine(
        multiCalc,
        AddressOf Subtract
    )
```

```
    Console.WriteLine("Multicast delegate:")
    multiCalc(10, 5)
```

```
End Sub
```

```
End Module
```

Working:

For:

```
a = 10
b = 5
```

Add:

$$10 + 5 = 15$$

Subtract:

$$10 - 5 = 5$$

Multiply:

$$10 \times 5 = 50$$

A multicast delegate contains more than one method, so calling it invokes all the methods attached to it.

n) Build a VB.NET code example showing: (4 marks)

- A class called TemperatureSensor that declares a custom event called TemperatureExceeded
- The event should pass a custom TemperatureEventArgs object containing CurrentTemperature and Threshold values
- A method that raises the event when temperature exceeds a threshold
- In a Windows Form, handle the event and display a warning message

Answer:**TemperatureEventArgs.vb**

```
Public Class TemperatureEventArgs  
    Inherits EventArgs
```

```
    Public Property CurrentTemperature As Double  
    Public Property Threshold As Double
```

```
    Public Sub New(currentTemperature As Double,  
        threshold As Double)
```

```
        Me.CurrentTemperature = currentTemperature  
        Me.Threshold = threshold
```

```
End Sub
```

```
End Class
```

TemperatureSensor.vb

```
Public Class TemperatureSensor
```

```
    Public Event TemperatureExceeded As _  
        EventHandler(Of TemperatureEventArgs)
```

```
    Public Sub CheckTemperature(currentTemperature As Double,  
        threshold As Double)
```

```
        If currentTemperature > threshold Then
```

```

        Dim e As New TemperatureEventArgs(
            currentTemperature,
            threshold
        )

        RaiseEvent TemperatureExceeded(Me, e)

    End If

End Sub

End Class

```

Windows Form

```

Public Class Form1

    Private sensor As New TemperatureSensor()

    Private Sub Form1_Load(sender As Object, e As EventArgs) _
        Handles MyBase.Load

        AddHandler sensor.TemperatureExceeded,
            AddressOf Sensor_TemperatureExceeded

    End Sub

    Private Sub Sensor_TemperatureExceeded(
        sender As Object,
        e As TemperatureEventArgs)

        MessageBox.Show(
            "WARNING!" & vbCrLf &
            "Current Temperature: " &
            e.CurrentTemperature & "°C" & vbCrLf &
            "Threshold: " &
            e.Threshold & "°C"
        )

    End Sub

    Private Sub btnCheck_Click(sender As Object, e As EventArgs) _
        Handles btnCheck.Click

        sensor.CheckTemperature(40, 35)

    End Sub

End Class

```

End Class

Working:

The values are:

Current Temperature = 40°C

Threshold = 35°C

Since:

40 > 35

the condition is true, so the event is raised:

```
TemperatureSensor
    ↓
TemperatureExceeded Event
    ↓
Event Handler
    ↓
Warning MessageBox
```

The warning displayed is approximately:

```
WARNING!
Current Temperature: 40°C
Threshold: 35°C
```

o) Explain the concept of garbage collection in .NET. Define the THREE phases of garbage collection and explain TWO ways to manage garbage collection in VB.NET. (3 marks)

Answer:

Garbage collection is the automatic process in .NET that identifies objects that are no longer being used and releases the memory occupied by them.

THREE phases of garbage collection:

1. **Marking** – The garbage collector identifies objects that are still being used.
2. **Relocating/Updating references** – References to live objects are updated when necessary.
3. **Compacting** – The GC moves live objects together and frees unused memory.

TWO ways to manage garbage collection:

1. **Use Dispose() / Using** for unmanaged or disposable resources such as files and database connections.

```
Using reader As New IO.StreamReader("data.txt")
    Console.WriteLine(reader.ReadLine())
End Using
```

2. **Use GC.Collect() carefully** to request garbage collection manually.

```
GC.Collect()
GC.WaitForPendingFinalizers()
```